



Security Review Report for IRIS

July 2026



Table of Contents

1. About Hexens
2. Executive summary
3. Security Review Details
 - Security Review Lead
 - Scope
 - Changelog
4. Severity Structure
 - Severity characteristics
 - Issue symbolic codes
5. Findings Summary
6. Weaknesses
 - Collateral withdrawal double-counts overdue interest
 - Bad debt close can route surviving surplus to the solver
 - Missing Mechanism to Claim Aave Incentive Rewards
 - Morpho bad debt socialization can lead to underflow reverts

1. About Hexens

Hexens is a pioneering cybersecurity firm dedicated to establishing robust security standards for Web3 infrastructure, driving secure mass adoption through innovative protection technology and frameworks. As an industry elite experts in blockchain security, we deliver comprehensive audit solutions across specialized domains, including infrastructure security, Zero Knowledge Proof, novel cryptography, DeFi protocols, and NFTs.

Our methodology combines industry-standard security practices combined with unique methodology of two teams per audit, continuously advancing the field of Web3 security. This innovative approach has earned us recognition from industry leaders.

Since our founding in 2021, we have built an exceptional portfolio of enterprise clients, including major blockchain ecosystems and Web3 platforms.

2. Executive Summary

This report covers the security review for Iris protocol, a non-custodial fixed-rate, fixed-term lending protocol. It makes use of variable-rate lending markets such as Morpho and Aave to create fixed-rate loans.

Our security assessment was a full review of the code, spanning a total of 1 week.

During our review, we did not identify any major severity vulnerabilities.

We did identify some minor severity vulnerabilities and code optimizations.

All reported issues were fixed or acknowledged by the development team and subsequently verified by us.

We can confidently say that the overall security and code quality have increased after completion of our audit.

3. Security Review Details

- **Review Led by**

Jahyun Koo, Lead Security Researcher

- **Scope**

The analyzed resources are located on:

<https://github.com/iris-credit/iris-core/tree/6b5b339036c82c45493ec0ff0ff3784faeb3e546>

The issues described in this report were fixed in the following commit:

<https://github.com/iris-credit/iris-core/tree/f2d9f3e96455f3524c2f92e14cdd1b25212c8b7d>

- **Changelog**

■ 6 July 2026	Audit start
■ 13 July 2026	Initial report
■ 15 July 2026	Revision received
■ 17 July 2026	Final report

4. Severity Structure

The vulnerability severity is calculated based on two components:

- 1. Impact of the vulnerability
- 2. Probability of the vulnerability

Impact	Probability			
	Rare	Unlikely	Likely	Very likely
Low	Low	Low	Medium	Medium
Medium	Low	Medium	Medium	High
High	Medium	Medium	High	Critical
Critical	Medium	High	Critical	Critical

▪ Severity Characteristics

Smart contract vulnerabilities can range in severity and impact, and it's important to understand their level of severity in order to prioritize their resolution. Here are the different types of severity levels of smart contract vulnerabilities:

Critical	Vulnerabilities that are highly likely to be exploited and can lead to catastrophic outcomes, such as total loss of protocol funds, unauthorized governance control, or permanent disruption of contract functionality.
High	Vulnerabilities that are likely to be exploited and can cause significant financial losses or severe operational disruptions, such as partial fund theft or temporary asset freezing.

Medium

Vulnerabilities that may be exploited under specific conditions and result in moderate harm, such as operational disruptions or limited financial impact without direct profit to the attacker.

Low

Vulnerabilities with low exploitation likelihood or minimal impact, affecting usability or efficiency but posing no significant security risk.

Informational

Issues that do not pose an immediate security risk but are relevant to best practices, code quality, or potential optimizations.

▪ Issue Symbolic Codes

Each identified and validated issue is assigned a unique symbolic code during the security research stage.

Due to the structure of the vulnerability reporting flow, some rejected issues may be missing.

5. Findings Summary

Severity

Number of findings

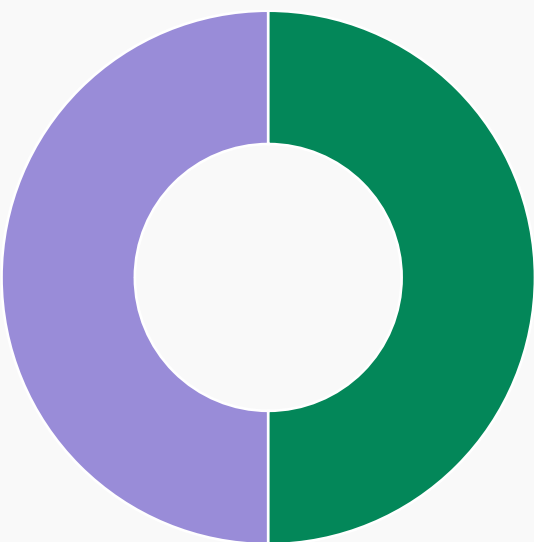
Critical	0
High	0
Medium	0
Low	4
Informational	0

Total:

4



Low



Fixed
Acknowledged

6. Weaknesses

This section contains the list of discovered weaknesses.

IRIS1-1 | Collateral withdrawal double-counts overdue interest

Fixed 

Severity:

Low

Probability:

Unlikely

Impact:

Low

Path:

iris-jul-26/src/Iris.sol:withdrawCollateral#L576-L611

Description:

The function **withdrawCollateral** lets the borrower withdraw collateral from their active loan pod. The calls into **_accrueLegs** which updates the loan values according to the current timestamp, such as accrue interest. The function then check that the remaining collateral after withdrawal covers the debt and finally sends out the assets.

The withdrawal is allowed when the remaining collateral covers the borrower's loan at **maturity + overduePeriod**. Because **withdrawCollateral** calls **_accrueLegs + _rebase** before performing the health check, **pos.fixedLeg** already includes fixed-rate interest up to the current timestamp which will also include any overdue interest.

However, the calculation in **withdrawCollateral** again includes the the full interest over the overdue period by taking **maturity + overduePeriod - block.timestamp**:

```
uint256 timeToLiquidation = uint256(loan.maturity + loan.overduePeriod).zeroFloorSub(block.timestamp);
uint256 residual = pos.debt
    .mulDivDown(
        timeToLiquidation * loan.fixedRate * BP + uint256(loan.overduePeriod) * loan.overdueRate * BP,
        SECONDS_PER_YEAR * WAD
    );

require(pos.debt + pos.fixedLeg + residual <= maxDebt, InsufficientCollateral());
```

The following require check takes both **pos.fixedLeg** and **residual**. Therefore it contains the interest over the overdue period twice. Once from **_accrueLegs + _rebase** in **pos.fixedLeg** and once from this calculation in **residual**.

Before maturity, no overdue interest has accrued and there is no overlap. After maturity, the elapsed portion of the overdue interest is included in both **pos.fixedLeg** and **residual**.

Consequently, the collateral check may reject a withdrawal that would otherwise satisfy the intended debt coverage requirement.

This makes the check stricter than necessary, allowing for less collateral to be withdrawn for an otherwise healthy loan.

```
function withdrawCollateral(address pod, uint256 amount, address receiver) external {
    Loan storage loan = _loan[pod];
    Position storage pos = _position[pod];
    address adapter = venueAdapter[pos.venueId];

    require(amount != 0, ZeroAmount());
    require(receiver != address(0), ZeroAddress());
    require(loan.collateralToken != address(0), ZeroAddress());
    require(!_isSenderAuthorized(loan.borrower), Unauthorized());

    _accrueLegs(loan, pos, pod);
    _rebase(loan, pos, pod);

    pos.collateral -= amount.toUint128();

    uint256 lltv = IVenueAdapter(adapter).lltv(loan.collateralToken, loan.debtToken, pos.data);
    uint256 collateralPrice = IVenueAdapter(adapter).price(loan.collateralToken, loan.debtToken, pos.data);
    uint256 maxDebt = pos.collateral.mulDivDown(collateralPrice,
    ORACLE_PRICE_SCALE).mulDivDown(lltv, WAD);
    uint256 timeToLiquidation = uint256(loan.maturity + loan.overduePeriod).zeroFloorSub(block.timestamp);
    uint256 residual = pos.debt
        .mulDivDown(
            timeToLiquidation * loan.fixedRate * BP + uint256(loan.overduePeriod) * loan.overdueRate * BP,
            SECONDS_PER_YEAR * WAD
        );

    require(pos.debt + pos.fixedLeg + residual <= maxDebt, InsufficientCollateral());

    IPod(pod)
        .delegateCall(
            adapter,
            abi.encodeWithSelector(
                IVenueAdapter.withdrawCollateral.selector, loan.collateralToken, amount, receiver, pos.data
            )
        );

    emit EventsLib.WithdrawCollateral(msg.sender, pod, receiver, amount);
}
```

Remediation:

Calculate the overdue component of residual using only the portion of the overdue period that has not yet accrued.

$\max(\text{maturity} + \text{overduePeriod} - \max(\text{now}, \text{maturity}), 0)$

Verification Notes:

The overdue component of the residual now uses only the portion of the overdue period that has not yet been accrued.

IRIS1-2 | Bad debt close can route surviving surplus to the solver

Fixed ✓

Severity:

Low

Probability:

Rare

Impact:

Low

Path:

src/Iris.sol#L890

Description:

When `_rebase` detects bad debt, it sets `pos.bondRequirement` to zero, but any surviving `pos.surplus` can remain on the position.

```
uint256 badDebt = venueDebt.zeroFloorSub(
    venueCollateral.mulDivDown(collateralPrice, ORACLE_PRICE_SCALE)
);

if (venueCollateral <= pos.surplus) pos.surplus =
venueCollateral.toUint128();
if (venueDebt <= pos.floatingLeg) pos.floatingLeg = venueDebt.toUint128();
if (badDebt != 0 || (venueDebt == 0 && venueCollateral == 0))
pos.bondRequirement = 0;
```

Later, `_settleLegs` credits any remaining surplus to the solver without checking whether the position was resolved as bad debt.

```
uint256 net = pos.fixedLeg > pos.floatingLeg ? pos.fixedLeg - pos.floatingLeg : 0;
uint256 _fee = loan.fee * BP;
uint256 performanceFee = net != 0 && _fee != 0 ? net.mulDivDown(_fee, WAD) : 0;
uint256 surplusFee = _fee != 0 ? pos.surplus.mulDivDown(_fee, WAD) : 0;

if (net != 0) _claimable(loan.debtToken, net - performanceFee, loan.solver);
if (pos.surplus != 0) _claimable(loan.collateralToken, pos.surplus - surplusFee, loan.solver);
if (performanceFee != 0) _claimable(loan.debtToken, performanceFee, feeRecipient);
if (surplusFee != 0) _claimable(loan.collateralToken, surplusFee, feeRecipient);
```

The contract documentation states that when `rebase` detects bad debt and sets `bondRequirement` to zero, the solver should not receive surplus or fixed interest.

```
/// @dev Surplus can be greater than the venue collateral in an extreme case where most of the collateral got  
/// liquidated in the underlying venue. In such a case, the surplus shrinks to venue collateral.  
/// @dev If rebase detects bad debt, bondRequirement is set to zero. In that state,  
/// the solver can withdraw bond even when net is negative but does not receive  
/// surplus or fixed interest because the loan is not expected to be repaid.
```

This creates a mismatch between the documented bad-debt behavior and the settlement implementation.

Remediation:

Gate solver surplus crediting in **_settleLegs** when the position has been resolved through bad debt.

Clear or separately preserve bad-debt surplus during **_rebase** so it is not routed through the normal solver settlement path.

Verification Notes:

The behavior regarding surplus claims for bad debt loans has been documented more clearly.

IRIS1-3 | Missing Mechanism to Claim Aave Incentive Rewards

Acknowledged

Severity:

Low

Probability:

Rare

Impact:

Low

Description:

Aave incentive rewards accrue to the position owner and must be claimed through the **RewardsController** by either the owner (**claimRewards**) or an authorized claimer (**claimRewardsOnBehalf**). However, pods cannot invoke either flow:

- The pod only executes logic exposed through approved adapters via **delegateCall**.
- The current adapter implementation does not expose any function that calls the Aave **RewardsController**.

As a result, any incentive rewards accrued by a pod remain unclaimable.

Remediation:

Implement an function that allows the pod to invoke **RewardsController.claimRewards()** (or **claimAllRewards()**) via **delegateCall**.

Verification Notes:

Aave incentives are now distributed offchain via Merkl. With development focusing on Aave v4, this is expected to remain the long term approach.

IRIS1-4 | Morpho bad debt socialization can lead to underflow reverts

Acknowledged

Severity:

Low

Probability:

Rare

Impact:

Medium

Path:

src/Iris.sol:_accrueLegsView#L849-L872

Description:

The function `_accrueLegsView` is the interest accrual helper used before most loan actions. Flows such as `repay`, `liquidate`, `withdrawBond`, `liquidateBond`, `refinance`, and `rebase` all call into `_accrueLegs` and then `_accrueLegsView` before any loan changes are made to accrue any loan value.

For a normal loan, this flow should be simple: the contracts reads the latest collateral and debt indices from the adapter and then calculates the new fixed and floating legs since the last update to store the fresh index snapshots. Afterwards it calls `_rebase` to further finalize any changes. If a Morpho position was liquidated and bad debt was detected, `_rebase` is also the path that can set `bondRequirement` to zero so the remaining bond and loan state can be settled cleanly.

That design relies on the debt index not moving below the stored `pos.debtIndex`. The code makes that assumption directly:

```
uint256 floatingLeg = (pos.debt + pos.floatingLeg).mulDivDown(newDebtIndex - pos.debtIndex, pos.debtIndex);
```

This is not safe for Morpho. `MorphoBlueAdapter.indices` returns the current market borrow share rate:

```
uint256 debtIndex = (totalBorrowAssets + SharesMathLib.VIRTUAL_ASSETS)
    .mulDivDown(INDEX_SCALE, totalBorrowShares + SharesMathLib.VIRTUAL_SHARES);
```

During a bad-debt liquidation, Morpho can socialize bad debt by removing from both `totalBorrowAssets` and `totalBorrowShares`. If the wiped borrower is a large part of the market, or the only borrower left, that ratio can fall a lot. For example, a market borrow index that had grown to around $1.10e27$ can fall back toward the empty-market value of $1e27$ after the real borrow side is wiped.

Once that happens, any loan in the same Morpho market whose stored **pos.debtIndex** is higher than the new adapter value will revert in **_accrueLegsView**. Checked arithmetic turns **newDebtIndex - pos.debtIndex** into an underflow revert. The important part is that this happens before **_rebase**, so it never gets to the code that would notice the bad debt.

The result is a persistent revert. The affected loan cannot be rebased until the index goes back to a value higher than the stored value. **escape** is also blocked while **bondRequirement** is still non-zero.

```
function _accrueLegsView(Loan storage loan, Position storage pos)
    internal
    view
    returns (uint256, uint256, uint256, uint256, uint256)
{
    uint256 elapsed = block.timestamp - pos.lastUpdate;
    if (pos.lastUpdate == 0 || elapsed == 0) return (pos.collateralIndex, pos.debtIndex, 0, 0, 0);

    (uint256 newCollateralIndex, uint256 newDebtIndex) =
        IVenueAdapter(venueAdapter[pos.venueId]).indices(loan.collateralToken, loan.debtToken, pos.data);
    uint256 fixedLeg = pos.debt.mulDivDown(elapsed * loan.fixedRate * BP, SECONDS_PER_YEAR * WAD);
    uint256 floatingLeg = (pos.debt + pos.floatingLeg).mulDivDown(newDebtIndex - pos.debtIndex,
pos.debtIndex);
    uint256 surplus = pos.bondRequirement != 0
        ? (pos.collateral + pos.surplus).mulDivDown(newCollateralIndex - pos.collateralIndex,
pos.collateralIndex)
        : 0;

    if (block.timestamp > loan.maturity) {
        uint256 overdueStart = MathLib.max(loan.maturity, pos.lastUpdate);
        uint256 overdueElapsed = block.timestamp - overdueStart;
        fixedLeg += pos.debt.mulDivDown(overdueElapsed * loan.overdueRate * BP, SECONDS_PER_YEAR
* WAD);
    }

    return (newCollateralIndex, newDebtIndex, fixedLeg, floatingLeg, surplus);
}
```

Remediation:

Do not require external indices to be monotonic before any bad debt calculation. We recommend to either perform a saturating subtraction such that any negative delta becomes 0 or to explicitly account them as a debt decrease.

Verification Notes:

A persistent underflow requires a pod to be essentially the entire borrow side of a market and be the socialized borrower. With any material remaining borrow the index deficit self heals via interest within seconds. Also Morpho markets are whitelisted blue chip markets where this extreme case cannot occur.

hexens x  IRIS

